

# Homework #2: Simple Neural Network Implementation using Numpy

Building a simple neural network from scratch using only NumPy.

## Step 1: Sigmoid Function and its Derivative

The **sigmoid** activation function is defined as:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Its derivative (used in backpropagation) is:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

### Proof

Let  $u(x) = 1 + e^{-x}$ , so  $\sigma(x) = u(x)^{-1}$  and  $u'(x) = -e^{-x}$ .

By the chain rule:

$$\frac{d\sigma(x)}{dx} = -u(x)^{-2} \cdot u'(x) = \frac{e^{-x}}{(1+e^{-x})^2}$$

Since  $1 - \sigma(x) = \frac{e^{-x}}{1+e^{-x}}$ , we get:

$$\sigma(x)(1 - \sigma(x)) = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \frac{e^{-x}}{(1+e^{-x})^2}$$

Therefore:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    # x here is the PRE-ACTIVATION value (before sigmoid)
    # We compute sigmoid(x) first, then apply the formula
```

```
sig = sigmoid(x)
return sig * (1 - sig)
```

## Step 2: Initialize Weights

We need a weight vector of shape  $(3 \times 1)$  because our input  $X$  has **3 features** and we predict **1 output**.

Weights are sampled from a normal distribution:

$$w \sim N(\mu=0, \sigma=1/3)$$

```
X = np.array([[0, 0, 1],
              [0, 1, 1],
              [1, 0, 1],
              [1, 1, 1]])

y = np.array([[0],
              [0],
              [1],
              [1]])

np.random.seed(42)
weights = np.random.normal(loc=0.0, scale=1/3, size=(3, 1))
print("Initial weights:\n", weights)
```

## Step 3: Training the Neural Network

Each training iteration performs:

**Forward pass:**

$$\hat{y} = \sigma(X \cdot w)$$

**Error:**

$$\delta = y - \hat{y}$$

**Backpropagation (gradient of loss w.r.t. weights):**

$$\nabla w = X^T \cdot (\delta \odot \sigma'(X \cdot w))$$

**Weight update:**

$$w \leftarrow w + \nabla w$$

**Note:**  $\sigma'$  is evaluated on the **pre-activation** value  $X \cdot w$ , not on  $\hat{y}$ .

```
for i in range(1000):
    # Forward pass
    pre_activation = np.dot(X, weights)    # shape: (4, 1)
```

```

output = sigmoid(pre_activation)          # shape: (4, 1)

# Error
error = y - output

# Backprop: use pre_activation (not output) in sigmoid_derivative
delta = error * sigmoid_derivative(pre_activation)

# Weight update
weights += np.dot(X.T, delta)

if i % 100 == 0:
    print(f"Iteration {i:4d} | MAE: {np.mean(np.abs(error)):.6f}")

print("\nFinal output:")
print(output)

```

## Results

After 1000 iterations the network correctly learns the mapping:

| $x_1$ | $x_2$ | $x_3$ | $\hat{y}$      | $y$ |
|-------|-------|-------|----------------|-----|
| 0     | 0     | 1     | $\approx 0.03$ | 0   |
| 0     | 1     | 1     | $\approx 0.03$ | 0   |
| 1     | 0     | 1     | $\approx 0.97$ | 1   |
| 1     | 1     | 1     | $\approx 0.97$ | 1   |

The network has learned that  $x_1$  is the relevant feature for predicting the output.